

PROMPT MAESTRO PARA CURSOR

Proyecto: Generador transversal de tutoriales web basado en eventos y semántica del DOM

Actúa como un equipo senior compuesto por:

- Arquitecto de software.
- Desarrollador TypeScript senior.
- Especialista en extensiones WebExtensions.
- Especialista en Chrome, Edge, Firefox y Safari.
- Desarrollador React.
- Especialista en accesibilidad web y semántica ARIA.
- Especialista en privacidad y protección de datos.
- Especialista en grabación y renderizado de tutoriales.

Tu misión es diseñar e implementar un MVP funcional para registrar la navegación de un usuario por un portal web y convertir sus acciones en un tutorial estructurado.

El sistema no debe depender de realizar una captura de pantalla por cada acción. Debe utilizar principalmente:

- Eventos del navegador.
- Análisis semántico del DOM.
- Información de accesibilidad.
- Posición visual de los elementos.
- Capturas estratégicas.
- Registro estructurado de pasos.
- Generación posterior de instrucciones narrativas.

1. OBJETIVO FUNCIONAL

Construir una solución que permita:

1. Iniciar una sesión de grabación.
2. Registrar las acciones realizadas por el usuario dentro de una página web.
3. Identificar semánticamente cada elemento utilizado.
4. Registrar clics, escritura, selecciones, envíos de formulario, scroll y navegación.
5. Detectar cambios relevantes en la interfaz.
6. Proteger automáticamente datos sensibles.
7. Guardar cada acción en un modelo JSON neutral.
8. Mostrar una lista editable de pasos.
9. Generar una narración inicial en español.
10. Exportar el tutorial como archivo JSON.
11. Reproducir visualmente el recorrido mediante resaltados y coordenadas.
12. Mantener compatibilidad con Chromium y Firefox desde una misma base TypeScript.

Safari debe quedar considerado arquitectónicamente, pero no es obligatorio empaquetarlo en la primera fase.

2. DECISIÓN ARQUITECTÓNICA

Implementa un monorepo con módulos separados.

Usa:

- TypeScript estricto.
- Node.js 20 o superior.
- pnpm workspaces.
- Vite.
- React.
- WebExtensions.
- `webextension-polyfill`.
- Zod para validación de contratos.
- Vitest para pruebas unitarias.
- ESLint.
- Prettier.

No agregues frameworks o dependencias que no sean necesarias.

Estructura inicial:

```
web-tutorial-recorder/  
├── apps/  
│   ├── extension/  
│   │   ├── src/  
│   │   │   ├── background/  
│   │   │   ├── content/  
│   │   │   ├── popup/  
│   │   │   ├── options/  
│   │   │   └── browser/  
│   │   ├── manifests/  
│   │   │   ├── chromium.json  
│   │   │   └── firefox.json  
│   │   └── vite.config.ts  
│   └── studio/  
│       ├── src/  
│       │   ├── components/  
│       │   ├── features/  
│       │   ├── pages/  
│       │   ├── services/  
│       │   └── stores/  
│       └── vite.config.ts
```

```

|
|— packages/
|   |— core/
|   |   |— src/
|   |   |   |— recorder/
|   |   |   |— semantic/
|   |   |   |— navigation/
|   |   |   |— privacy/
|   |   |   |— selectors/
|   |   |   └─ index.ts
|   |
|   |— contracts/
|   |   └─ src/
|   |       |— action.schema.ts
|   |       |— session.schema.ts
|   |       |— message.schema.ts
|   |       └─ index.ts
|   |
|   └─ tutorial-engine/
|       └─ src/
|           |— normalize-actions.ts
|           |— group-actions.ts
|           |— narrative-generator.ts
|           └─ index.ts
|
|— docs/
|   |— ARCHITECTURE.md
|   |— PRIVACY.md
|   |— EVENT-MODEL.md
|   └─ ROADMAP.md
|
|— pnpm-workspace.yaml
|— tsconfig.base.json
|— package.json
└─ README.md

```

3. PRINCIPIOS OBLIGATORIOS

3.1 Código transversal

La lógica central no debe depender de:

- `chrome.*`
- APIs exclusivas de Chromium.
- Chrome DevTools Protocol.
- Selectores propietarios.
- APIs experimentales sin fallback.

Usa `webextension-polyfill` para comunicar:

- Content script.
- Background service.
- Popup.
- Página de opciones.

Cualquier diferencia entre navegadores debe aislarse en adaptadores:

```
browser/  
├─ browser-adapter.ts  
├─ chromium-adapter.ts  
└─ firefox-adapter.ts
```

3.2 Separación de responsabilidades

El núcleo `packages/core` no debe depender de React ni de las APIs de extensión.

Debe ser ejecutable y comprobable mediante pruebas unitarias.

3.3 TypeScript estricto

Activa como mínimo:

```
{  
  "compilerOptions": {  
    "strict": true,  
    "noUncheckedIndexedAccess": true,  
    "exactOptionalPropertyTypes": true,  
    "noImplicitOverride": true,  
    "useUnknownInCatchVariables": true  
  }  
}
```

No utilices `any`, salvo que exista una justificación explícita y documentada.

4. MODELO DE DATOS

Define contratos con TypeScript y Zod.

El contrato principal debe ser equivalente a:

```
export type TutorialActionType =  
  | "click"  
  | "doubleClick"
```

```

    | "input"
    | "change"
    | "submit"
    | "navigation"
    | "scroll"
    | "keyboard"
    | "interfaceChange";

export interface BoundingBox {
  x: number;
  y: number;
  width: number;
  height: number;
  top: number;
  right: number;
  bottom: number;
  left: number;
}

export interface ViewportState {
  width: number;
  height: number;
  devicePixelRatio: number;
  scrollX: number;
  scrollY: number;
}

export interface SemanticTarget {
  tagName: string;
  role?: string;
  accessibleName?: string;
  text?: string;
  ariaLabel?: string;
  label?: string;
  placeholder?: string;
  title?: string;
  name?: string;
  inputType?: string;
  closestHeader?: string;
  formContext?: string;
  sectionContext?: string;
  breadcrumbContext?: string;
  testId?: string;
  href?: string;
  locatorCandidates: LocatorCandidate[];
  boundingBox?: BoundingBox;
}

export interface LocatorCandidate {
  strategy:
    | "testId"

```

```

    | "roleAndName"
    | "label"
    | "placeholder"
    | "text"
    | "id"
    | "name"
    | "css";
value: string;
score: number;
}

export interface TutorialAction {
  id: string;
  sessionId: string;
  sequence: number;
  type: TutorialActionType;
  timestamp: number;
  url: string;
  pageTitle: string;
  target?: SemanticTarget;
  value?: string;
  previousValue?: string;
  sensitive: boolean;
  viewport: ViewportState;
  metadata: {
    browser?: string;
    frameId?: string;
    source: "content-script" | "background" | "history-hook";
  };
}

export interface TutorialSession {
  id: string;
  name: string;
  status: "idle" | "recording" | "paused" | "completed";
  startedAt: number;
  endedAt?: number;
  initialUrl: string;
  actions: TutorialAction[];
}

```

Utiliza UUID generado con `crypto.randomUUID()`.

No instales una dependencia solo para generar UUID.

5. CAPTURA DE EVENTOS

Implementa un `EventRecorder` independiente.

Debe capturar mediante event delegation:

- `click`
- `dblclick`
- `input`
- `change`
- `submit`
- `keydown`
- `scroll`

Usa listeners en fase de captura cuando corresponda.

Ejemplo conceptual:

```
document.addEventListener("click", handleClick, true);
```

Usa `event.composedPath()` para soportar Shadow DOM abierto.

No registres indiscriminadamente todos los eventos de teclado.

Solo registra teclas que representen una acción relevante:

- Enter.
- Escape.
- Tab cuando cambie el contexto.
- Atajos configurables.
- Activación de botones mediante teclado.

No guardes teclas individuales escritas en campos de texto.

Para campos de texto:

- Agrupa eventos consecutivos.
- Aplica debounce.
- Genera una sola acción después de una pausa razonable.
- Nunca registres cada carácter como un paso independiente.

6. ANALIZADOR SEMÁNTICO

Implementa una función:

```
analyzeElement(element: Element): SemanticTarget
```

Debe priorizar los metadatos en este orden:

1. Nombre accesible.

2. `aria-label`.
3. `aria-labelledby`.
4. Etiqueta `<label>` asociada.
5. Rol semántico.
6. Texto visible.
7. Placeholder.
8. Atributo `title`.
9. Nombre del campo.
10. Encabezado cercano.
11. Formulario contenedor.
12. Sección contenedora.
13. Breadcrumb.
14. Selector estable como último recurso.

Implementa funciones pequeñas y comprobables:

```
getAccessibleName()
inferRole()
getAssociatedLabel()
getVisibleText()
findClosestHeader()
findFormContext()
findSectionContext()
findBreadcrumbContext()
generateLocatorCandidates()
getBoundingBox()
```

No uses únicamente `innerText`.

Normaliza espacios y elimina texto excesivo.

Limita el texto capturado a una cantidad razonable de caracteres.

No captures el contenido completo de contenedores grandes.

7. GENERACIÓN DE LOCALIZADORES

Genera varios candidatos y asigna una puntuación de estabilidad.

Orden sugerido:

1. `data-testid`, `data-test`, `data-qa`.
2. Rol y nombre accesible.
3. Label asociado.
4. Placeholder.
5. ID estable.
6. Name estable.

7. Texto visible corto.
8. CSS estructural como fallback.

Evita como primera opción:

```
div:nth-child(4) > div:nth-child(2) > button
```

Detecta IDs probablemente dinámicos, por ejemplo:

- UUID.
- Hashes largos.
- Secuencias numéricas variables.
- Identificadores generados por frameworks.

Cada localizador debe tener un score entre 0 y 100.

8. PRIVACIDAD Y DATOS SENSIBLES

Implementa un `PrivacyFilter`.

Debe marcar o anonimizar:

- Campos `type="password"`.
- Números de tarjetas.
- Códigos CVV.
- Tokens.
- Claves API.
- Secretos.
- Campos de autenticación.
- RUT.
- Correos electrónicos, configurable.
- Teléfonos, configurable.
- Datos definidos mediante selectores por el usuario.

Reglas obligatorias:

1. Una contraseña nunca puede almacenarse.
2. El valor de un campo sensible debe reemplazarse por `[REDACTED]`.
3. El sistema puede conservar la longitud de forma opcional, pero no el contenido.
4. La detección debe usar atributos, labels, nombres, placeholders y patrones.
5. Debe existir una lista configurable de expresiones sensibles.
6. Las pruebas deben verificar que no se filtren secretos.

Ejemplo:

```
{  
  value: "[REDACTED]",  
}
```

```
sensitive: true
}
```

9. NAVEGACIÓN EN APLICACIONES SPA

Detecta navegación tradicional y SPA.

Interpreta:

- `popstate`.
- `hashchange`.
- Cambios de `location.href`.
- `history.pushState`.
- `history.replaceState`.

Implementa un hook seguro para `history`, preservando el comportamiento original.

No generes varias acciones de navegación para el mismo cambio de URL.

Aplica deduplicación por URL y ventana temporal.

10. DETECCIÓN DE CAMBIOS EN LA INTERFAZ

Usa `MutationObserver`, pero no guardes cada mutación.

Implementa:

- Debounce.
- Agrupación de cambios.
- Filtro de mutaciones irrelevantes.
- Detección de aparición de modal.
- Detección de mensajes de éxito o error.
- Detección de cambios visibles relevantes.
- Detección de nuevos encabezados o secciones.

Genera una acción `interfaceChange` solo cuando el cambio aporte contexto al tutorial.

No registres mutaciones producidas por la propia extensión.

Marca los elementos visuales de la extensión con un atributo reservado, por ejemplo:

```
data-tutorial-recorder-ui="true"
```

Ignora esos elementos en el observador y en el capturador.

11. IFRAMES Y SHADOW DOM

Para el MVP:

- Soporta Shadow DOM abierto mediante `composedPath()`.
- Documenta las limitaciones de Shadow DOM cerrado.
- Inyecta content scripts en todos los frames cuando los permisos lo permitan.
- Registra un identificador de frame.
- No intentes acceder a iframes de origen cruzado directamente desde la página principal.
- Usa la infraestructura de la extensión para recibir eventos desde cada frame.

Documenta claramente las limitaciones de:

- Iframes cross-origin.
- Canvas.
- WebGL.
- Escritorios remotos.
- Citrix.
- Aplicaciones que no exponen semántica DOM.

12. MENSAJERÍA INTERNA

Define mensajes validados con Zod.

Como mínimo:

```
type ExtensionMessage =
| { type: "START_RECORDING"; payload: StartRecordingPayload }
| { type: "PAUSE_RECORDING" }
| { type: "RESUME_RECORDING" }
| { type: "STOP_RECORDING" }
| { type: "RECORDED_ACTION"; payload: TutorialAction }
| { type: "GET_SESSION" }
| { type: "SESSION_UPDATED"; payload: TutorialSession }
| { type: "EXPORT_SESSION" };
```

Nunca confíes directamente en mensajes recibidos.

Valida todo payload antes de procesarlo.

13. ALMACENAMIENTO

Para el MVP usa almacenamiento local de la extensión.

Implementa una abstracción:

```
interface SessionRepository {  
  getCurrent(): Promise<TutorialSession | null>;  
  save(session: TutorialSession): Promise<void>;  
  clear(): Promise<void>;  
  export(sessionId: string): Promise<string>;  
}
```

No acoples el núcleo a `browser.storage`.

Crea un adaptador específico dentro de la extensión.

Considera límites de tamaño.

No guardes capturas completas en el almacenamiento de sincronización.

14. INTERFAZ DE LA EXTENSIÓN

Construye un popup React sencillo.

Debe mostrar:

- Estado actual.
- Botón "Iniciar grabación".
- Botón "Pausar".
- Botón "Reanudar".
- Botón "Finalizar".
- Cantidad de acciones registradas.
- URL actual.
- Botón "Abrir estudio".
- Botón "Exportar JSON".

Los botones deben habilitarse según el estado.

Ejemplo:

```
Estado: Grabando  
Página: https://portal.ejemplo.cl/usuarios  
Pasos registrados: 8  
  
[Pausar] [Finalizar]  
[Abrir estudio]
```

La interfaz debe ser accesible mediante teclado.

15. ESTUDIO DEL TUTORIAL

Implementa una aplicación React independiente llamada `studio`.

Inicialmente puede cargar un JSON exportado mediante selector de archivo.

Debe mostrar:

- Nombre del tutorial.
- URL inicial.
- Fecha.
- Lista ordenada de pasos.
- Tipo de acción.
- Descripción generada.
- Contexto semántico.
- Campo sensible oculto.
- Posición del elemento.
- Mejor localizador.
- Botones para editar, eliminar y reordenar pasos.

No implementes todavía un editor de video avanzado.

Incluye una vista previa simple del tutorial.

16. MOTOR DE NARRACIÓN

Implementa una narración determinista sin IA como primera versión.

Función principal:

```
generateNarrativeStep(action: TutorialAction): string
```

Ejemplos:

- Click sobre botón “Nuevo usuario”:

```
Haz clic en “Nuevo usuario”.
```

- Campo con label “Correo electrónico”:

```
Ingresa el correo electrónico.
```

- Select “Tipo de usuario”:

Selecciona una opción en “Tipo de usuario”.

- Submit del formulario “Nuevo usuario”:

Una vez completados los datos, envía el formulario “Nuevo usuario”.

- Navegación:

A continuación, accede a la sección “Usuarios”.

Genera una narración general usando conectores:

- Primero.
- Luego.
- A continuación.
- Después.
- Una vez completado.
- Finalmente.

No menciones:

- DOM.
- CSS.
- XPath.
- Selectores.
- Playwright.
- TagName.
- Coordenadas.
- Detalles técnicos.

El resultado debe estar listo para ser leído por una voz o avatar.

17. AGRUPACIÓN Y NORMALIZACIÓN

Implementa un pipeline:

```
acciones crudas
↓
deduplicación
↓
normalización
↓
agrupación
↓
eliminación de ruido
↓
```

generación de pasos

↓

narración

Reglas iniciales:

1. Agrupar múltiples eventos `input` sobre el mismo campo.
2. Eliminar clic seguido inmediatamente por navegación cuando el clic ya explica la acción.
3. Conservar la navegación como resultado o contexto, no necesariamente como paso visible.
4. Agrupar scroll continuo.
5. Ignorar scroll mínimo.
6. Deduplicar eventos idénticos dentro de una ventana temporal.
7. Mantener submit y resultado de éxito como pasos relacionados.
8. No inventar intenciones que no puedan deducirse.

18. RESALTADO VISUAL

Implementa un overlay ligero durante la grabación.

Debe poder:

- Dibujar un borde alrededor del último elemento accionado.
- Mostrar un indicador de grabación.
- No bloquear la interacción.
- No contaminar la aplicación host.
- Usar Shadow DOM para aislar estilos cuando sea posible.
- Tener `pointer-events: none`.
- Usar un `z-index` elevado, pero documentado.
- Ser removible al finalizar.

No uses estilos globales.

No modifiques permanentemente el DOM del portal.

19. MANIFESTS

Genera:

```
manifest.chromium.json  
manifest.firefox.json
```

Solicita únicamente los permisos necesarios.

Prioriza:

- `storage`
- `activeTab`
- `scripting`
- `tabs`, solo si es realmente necesario

Evita pedir acceso global permanente si puede utilizarse un flujo de autorización explícito.

Para desarrollo se puede usar:

```
{  
  "host_permissions": ["<all_urls>"]  
}
```

Pero deja documentado que producción debe restringir permisos o utilizar permisos opcionales.

Configura content scripts para frames cuando corresponda.

20. PRUEBAS

Crea pruebas unitarias para:

- Nombre accesible.
- Label asociado.
- Inferencia de rol.
- Encabezado cercano.
- Generación de localizadores.
- Detección de ID dinámico.
- Redacción de contraseñas.
- Redacción de tarjetas.
- Agrupación de eventos input.
- Deduplicación.
- Narración de clic.
- Narración de input.
- Narración de navegación.
- Validación de mensajes.

Usa fixtures HTML pequeños con jsdom.

No escribas pruebas que dependan de servicios externos.

21. DOCUMENTACIÓN

Genera:

README.md

Debe incluir:

- Objetivo.
- Arquitectura.
- Instalación.
- Ejecución.
- Compilación.
- Carga de extensión en Chrome.
- Carga temporal en Firefox.
- Flujo de grabación.
- Exportación del tutorial.
- Limitaciones conocidas.

ARCHITECTURE.md

Debe explicar:

- Núcleo compartido.
- Comunicación entre procesos.
- Flujo de eventos.
- Modelo de datos.
- Aislamiento por navegador.
- Decisiones arquitectónicas.

PRIVACY.md

Debe explicar:

- Qué datos se capturan.
- Qué datos no se deben guardar.
- Redacción.
- Consentimiento.
- Riesgos.
- Recomendaciones para uso empresarial.

ROADMAP.md

Debe contemplar:

- Capturas estratégicas.
- Grabación de pestaña.
- Backend.
- Autenticación.
- Sincronización.
- IA para generación del guion.
- Voz sintética.
- Remotion.
- FFmpeg.

- Exportación MP4.
 - Safari.
 - Editor visual.
 - Colaboración.
-

22. REGLAS DE IMPLEMENTACIÓN PARA CURSOR

Debes trabajar por etapas pequeñas y verificables.

No implementes todo de una sola vez.

Antes de escribir código:

1. Inspecciona el repositorio.
2. Informa brevemente qué archivos crearás o modificarás.
3. Identifica riesgos o contradicciones.
4. Propón el siguiente bloque pequeño de trabajo.

Después implementa solamente la fase solicitada.

No modifiques archivos no relacionados.

No refactorices código existente fuera del alcance.

No cambies nombres públicos sin necesidad.

No dupliques lógica.

No crees abstracciones prematuras.

No agregues dependencias sin explicar:

- Para qué se usan.
- Por qué son necesarias.
- Qué alternativa nativa se descartó.

Después de cada fase:

1. Ejecuta TypeScript.
2. Ejecuta lint.
3. Ejecuta pruebas.
4. Corrige los errores provocados por tus cambios.
5. Resume los archivos modificados.
6. Indica lo que falta.
7. Detente y espera la siguiente instrucción.

No afirmes que algo funciona si no lo verificaste.

No uses pseudocódigo en archivos de producción.

No dejes funciones vacías simulando funcionalidad terminada.

Los TODO deben representar trabajo futuro real y deben estar documentados.

23. FASES DE IMPLEMENTACIÓN

Fase 1 — Fundaciones

Implementa solamente:

- Monorepo.
- Configuración TypeScript.
- pnpm workspaces.
- ESLint.
- Prettier.
- Vitest.
- Paquete `contracts`.
- Contratos Zod.
- README inicial.

Criterios de aceptación:

- `pnpm install` funciona.
- `pnpm typecheck` funciona.
- `pnpm test` funciona.
- Los contratos tienen pruebas.
- No existe código de extensión todavía.

Fase 2 — Núcleo semántico

Implementa:

- `analyzeElement`.
- Nombre accesible básico.
- Label asociado.
- Texto visible.
- Rol.
- Encabezado cercano.
- Form context.
- Bounding box.
- Candidatos de localizador.
- Pruebas.

Fase 3 — Privacidad

Implementa:

- Clasificación de campos sensibles.

- Redacción.
- Configuración.
- Pruebas de seguridad.

Fase 4 — Grabador de eventos

Implementa:

- EventRecorder.
- Click.
- Input con debounce.
- Change.
- Submit.
- Scroll agrupado.
- Ciclo start, pause, resume y stop.
- Pruebas.

Fase 5 — Extensión Chromium

Implementa:

- Manifest.
- Content script.
- Background.
- Popup.
- Mensajería.
- Storage.
- Exportación JSON.

Fase 6 — Firefox

Implementa:

- Manifest Firefox.
- Adaptadores.
- Build separado.
- Documentación de instalación.

Fase 7 — Navegación y cambios de interfaz

Implementa:

- History hook.
- SPA.
- MutationObserver.
- Detección de modal y mensajes.

Fase 8 — Tutorial Engine

Implementa:

- Normalización.
- Agrupación.
- Deduplicación.
- Narración determinista.
- Pruebas.

Fase 9 — Studio React

Implementa:

- Importación JSON.
- Lista de pasos.
- Edición.
- Eliminación.
- Reordenamiento.
- Exportación corregida.

Fase 10 — Vista previa

Implementa:

- Resaltado usando bounding boxes.
- Cursor simulado.
- Secuencia de pasos.
- Control básico de reproducción.

24. PRIMERA INSTRUCCIÓN

Comienza únicamente con la Fase 1.

No avances a la Fase 2.

Primero inspecciona la carpeta actual.

Si está vacía, crea la estructura mínima.

Si contiene un proyecto existente, adapta la propuesta sin destruirlo.

Entrega código real y ejecutable.

Al terminar:

- Ejecuta las validaciones.

- Muestra un resumen conciso.
- Enumera los comandos disponibles.
- Indica cualquier limitación encontrada.
- Detente.